AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 3: Building AI Workflows with MCP

Understanding the Limits of MCP

Run the MCP-only review several times and let it act, to see what access alone doesn't solve

In the last two videos, MCP gave the agent fresh, traceable data and an answer that changed when the system did. In this video you test the MCP-only workflow the way you tested the Skill in Chapter 2: repeat it, compare the results, and let it take an action. Then you'll identify which challenges MCP doesn't solve on its own.

Before you start

You need the `orbit` server connected and the Skill still parked, as at the end of video 03_01. The TCK-2031 demo ticket from video 03_02 can stay in the data. If you're not sure about your workspace, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 03_01
```

Start a new Claude Code session in `orbit-launch`.

Step 1: Run the same request three times

Send this request, the same one you used in video 03_02:

```
Using only the orbit server, review the Orbit 2.0 launch and tell me the
biggest risks.
```

When the report is done, type `/clear` and send the same request again. Do this three times in total. Clearing between runs means each one starts fresh, like three different teammates asking.

Step 2: Compare the three runs

Compare your three runs, and the report you saved in `reports/mcp-only.md`, using these questions:

1. Is the report structured the same way: the same headings, and a table or list in the same shape?

2. Are the same words used for severity (High, Medium, Low), or do labels like Critical or Major appear?
3. Is the PayGate slip rated the same way every time, with the date math shown?
4. Is the typo ticket (TCK-2005) left out every time?
5. Is the ticket waiting on the customer (TCK-2016) left out every time?
6. Does every risk include evidence?

The data was the same every time. Expect at least some of the answers to be no.

Step 3: Let the agent record the risks

Without clearing the conversation after your third run, send:

```
Record these risks in the launch risk register.
```

Approve the `create_risk_item` calls, and note what gets recorded:

- Which severities were recorded? Only High risks, or Medium and Low too?
- How are the titles worded, and does each item include evidence?

Approve these calls only in your local course data. With a real system, think carefully before approving a tool that writes to something your whole team uses.

Step 4: Run it again and check the register

Type `/clear`, send the review request again, then ask it to record the risks again:

```
Using only the orbit server, review the Orbit 2.0 launch and tell me the
biggest risks.
Record these risks in the launch risk register.
```

Then check the register:

```
List everything in the launch risk register.
```

Look for duplicates: the same underlying risk recorded twice, possibly with different titles or severities. Nothing told the agent to check what was already in the register before writing. If your agent did check this time, notice that it was the agent's choice in that run, not a rule it will always follow.

Step 5: Reset the data

Clean up before moving on. From a terminal in `orbit-backend`, run:

```
npm run reset
```

This clears the risk register and removes the demo ticket, so Chapter 4 starts from a clean state.

What you just saw

Every run had the same access to the same data. What changed was the judgment:

- **Know-how.** Orbit Labs' rules (more than two weeks of delay is High, ignore cosmetic issues, don't count tickets waiting on the customer) weren't applied, because the agent had no way to know them.
- **Consistency.** The structure, the severity labels, and even which items counted as risks changed from run to run.
- **Safe use of write tools.** The agent could write to the register, but nothing told it what qualifies or to check for existing items first.

Other challenges MCP doesn't solve on its own

- **Data quality.** MCP returns what the system holds. Milestone M4 is marked "on track" even though its docs task is 35% done and the writer is out for a week. Deciding whether to trust a status field is judgment, not access.
- **Trust and security.** Real MCP servers need authentication, and you should grant only the access a workflow needs. Treat data returned by tools as information, not instructions: a ticket written by a customer shouldn't be able to change what the agent does.
- **Tool descriptions aren't the place for process.** It can be tempting to add your team's rules to a tool's description. That ties the rules to one tool and one server, and makes them hard to maintain.

The pattern

Compare the two halves of the course so far:

- **Skill only:** consistent judgment and format, but stale data and no actions.
- **MCP only:** fresh data and actions, but inconsistent judgment and no rules for writing.

Each one is strong exactly where the other is weak. MCP gives the agent access. It doesn't give it your team's judgment. That's why Chapter 4 brings the two together.